

EVPitch Technical Handover

System: EVPitch recruitment platform

Document version: 1.0

Prepared: 9 September 2026

Status: Current production handover

Contents

- | | |
|---------------------------------|---|
| 1. Purpose of this document | 12. Scheduled and startup work |
| 2. System overview | 13. Environment configuration |
| 3. High-level architecture | 14. Local development |
| 4. Repository layout | 15. Build and test commands |
| 5. Main functional areas | 16. Deployment and release procedure |
| 6. Job-post pricing and credits | 17. Backup and recovery |
| 7. Refund rules | 18. Monitoring and operational checks |
| 8. Payments | 19. Security notes and known technical debt |
| 9. API conventions | 20. Support runbook |
| 10. Data model summary | 21. Acceptance record for the September 2026 pricing work |
| 11. Storage and media | 22. Ownership and handover checklist |

1. Purpose of this document

This document explains how the EVPitch platform is structured, how its main services work together, and how the system should be operated and maintained. It is intended for the client, future developers, system administrators, and support staff.

The handover covers the public website, administration portal, backend API, mobile application, database, payments, job-post credits, video storage, email, real-time messaging, deployment, testing, and routine support.

No passwords, API secrets, private keys, or customer data are included. Those items must remain in the relevant server environment or approved password manager.

2. System overview

EVPitch is a recruitment platform for four user groups:

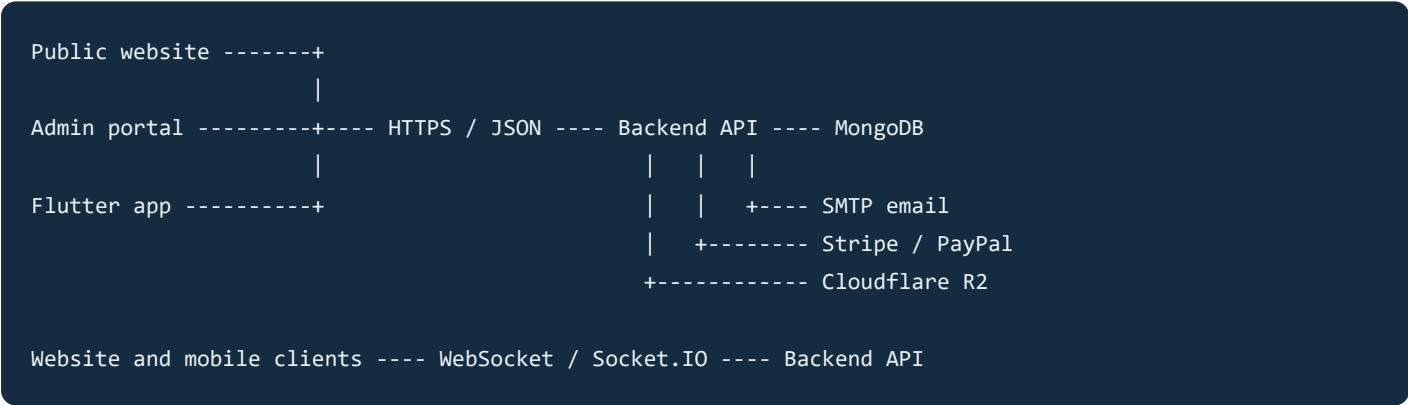
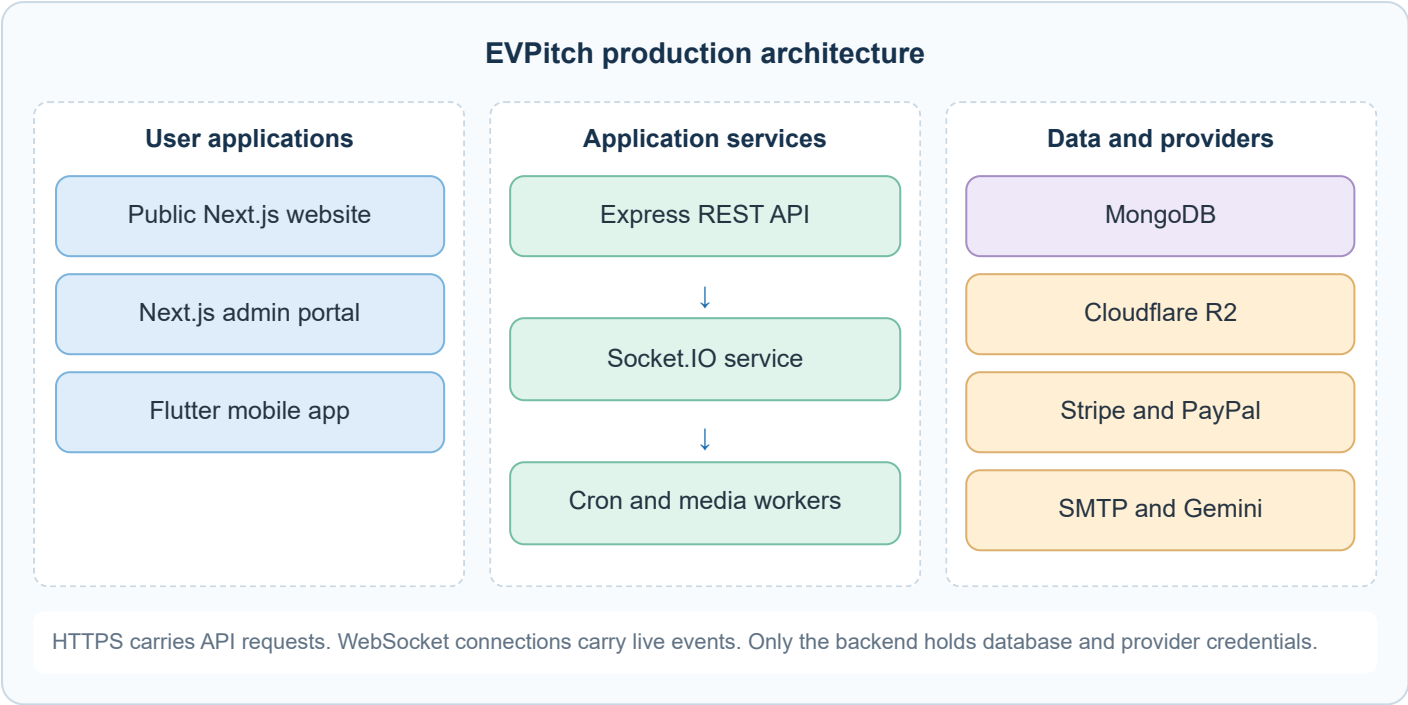
- Candidates create profiles, resumes, and elevator video pitches, search for jobs, and submit applications.
- Recruiters maintain recruiter profiles, work independently or in association with a company, post jobs, and manage applicants.
- Companies maintain company profiles, manage recruiters, publish jobs, and review applicants.
- Administrators manage users, jobs, plans, payments, content, FAQs, blogs, skills, newsletters, chatbot material, and scrolling messages.

The platform is divided into four codebases.

Application	Directory	Technology	Responsibility
Public website	evpitch-frontend	Next.js 15, React 18, TypeScript	Public pages, user accounts, profiles, jobs, checkout, messaging, and dashboards
Admin portal	evpitch-admin	Next.js 15, React 18, TypeScript	Operational and content administration
Backend API	evpitch-backend	Node.js, Express 5, TypeScript, Mongoose	Business rules, authentication, persistence, payments, media, email, scheduled work, and sockets
Mobile app	flutter_giveandtake_getx	Flutter 3 / Dart, GetX	Candidate, recruiter, and company mobile workflows

The production API is served under `/api/v1`. The backend also exposes `/` and `/health` for service checks.

3. High-level architecture



The public website and admin portal use NextAuth credential sessions. Login credentials are sent to the EVPitch API, which returns a JWT access token. Protected API requests then send that token as `Authorization: Bearer <token>`.

The Flutter application calls the same API and stores authentication material using secure local storage. Both web and mobile clients connect to Socket.IO for message and notification updates.

4. Repository layout

Backend

Path	Purpose
<code>src/app.ts</code>	Express application, middleware, health routes, and API route registration
<code>src/server.ts</code>	Database startup, HTTP/Socket.IO server, boot tasks, and cron schedules
<code>src/routes</code>	HTTP route declarations

Path	Purpose
<code>src/controllers</code>	Request handling and business workflows
<code>src/models</code>	Mongoose schemas and indexes
<code>src/services</code>	Payments, credits, storage, video, chatbot, embeddings, and deletion services
<code>src/middlewares</code>	JWT authentication, role checks, uploads, access gates, errors, and 404 handling
<code>src/jobs</code>	Startup consistency checks and scheduled maintenance
<code>src/scripts</code>	Operational migrations and search-index utilities
<code>src/utils</code>	Shared policy, response, email, token, pagination, and media helpers
<code>test</code>	Node test runner suites

Public website

The website uses the Next.js App Router. Route groups under `app/(auth)` contain sign-in and recovery journeys. User-facing routes are under `app/(website)`. Shared controls are under `components`, API access is under `lib`, and authentication-aware reusable behaviour is under `hooks`.

Admin portal

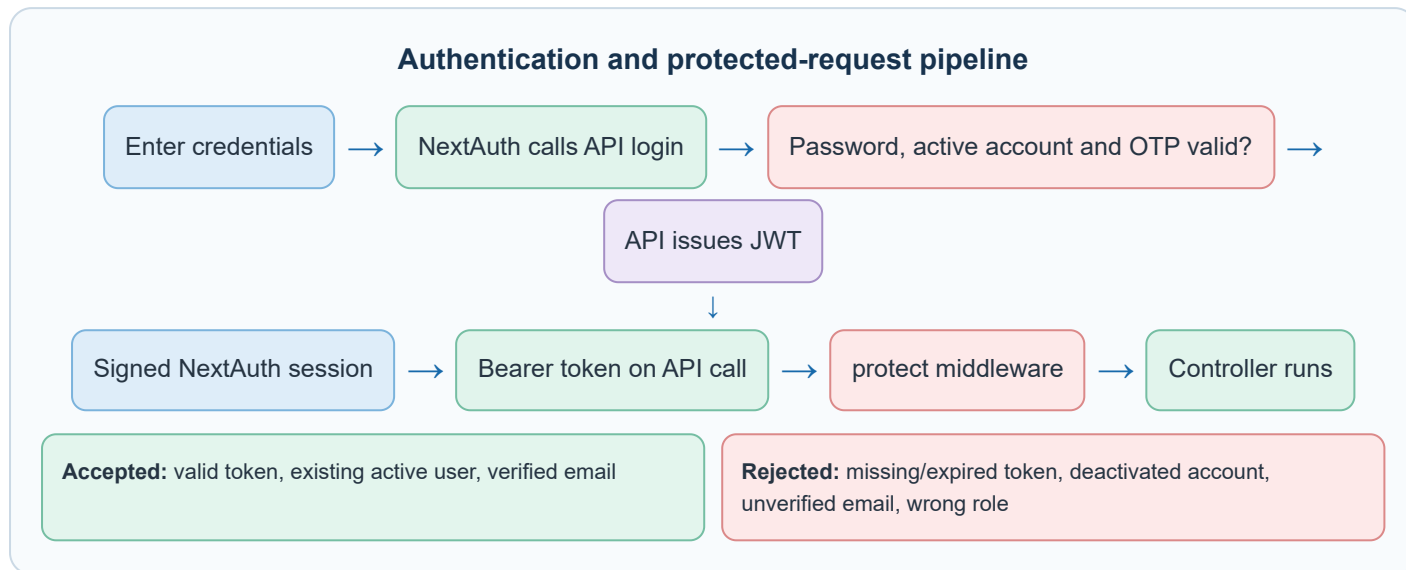
The admin portal also uses the App Router. Administrative pages are under `src/app/(admin-dashboard)`, sign-in and account recovery are under `src/app/(auth)`, and shared components and utilities are under `src/components` and `src/lib`.

Flutter application

The mobile application is organised by feature under `lib/features`. Shared API, socket, storage, and dependency-injection code is under `lib/core`. The production API and website addresses are currently declared in `lib/core/network/constants/api_constants.dart`.

5. Main functional areas

Accounts and access

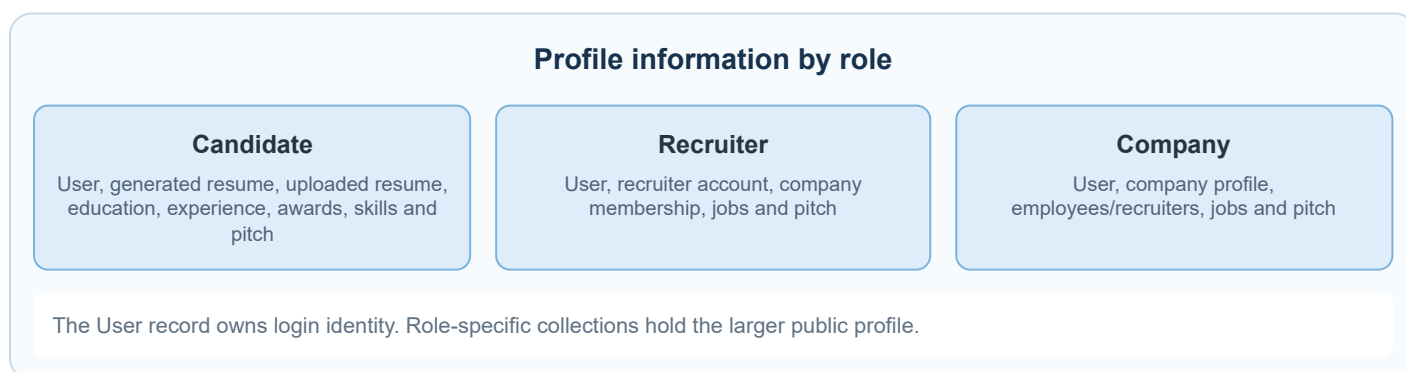


The user record supports `candidate`, `recruiter`, `company`, `admin`, and `super-admin` roles. Registration includes email verification by OTP. The account service also supports password reset, security questions, email changes, access-token refresh, and soft or scheduled account deactivation.

Protected backend routes use the `protect` middleware. It validates the JWT, confirms that the user still exists, rejects deactivated accounts, and checks email verification. Administrative catalogue and content operations add the `isAdmin` role check.

The admin portal only accepts staff accounts and enforces a 30-minute inactivity window through middleware. The public website protects selected account areas with NextAuth middleware; individual pages and API requests also apply their own session checks.

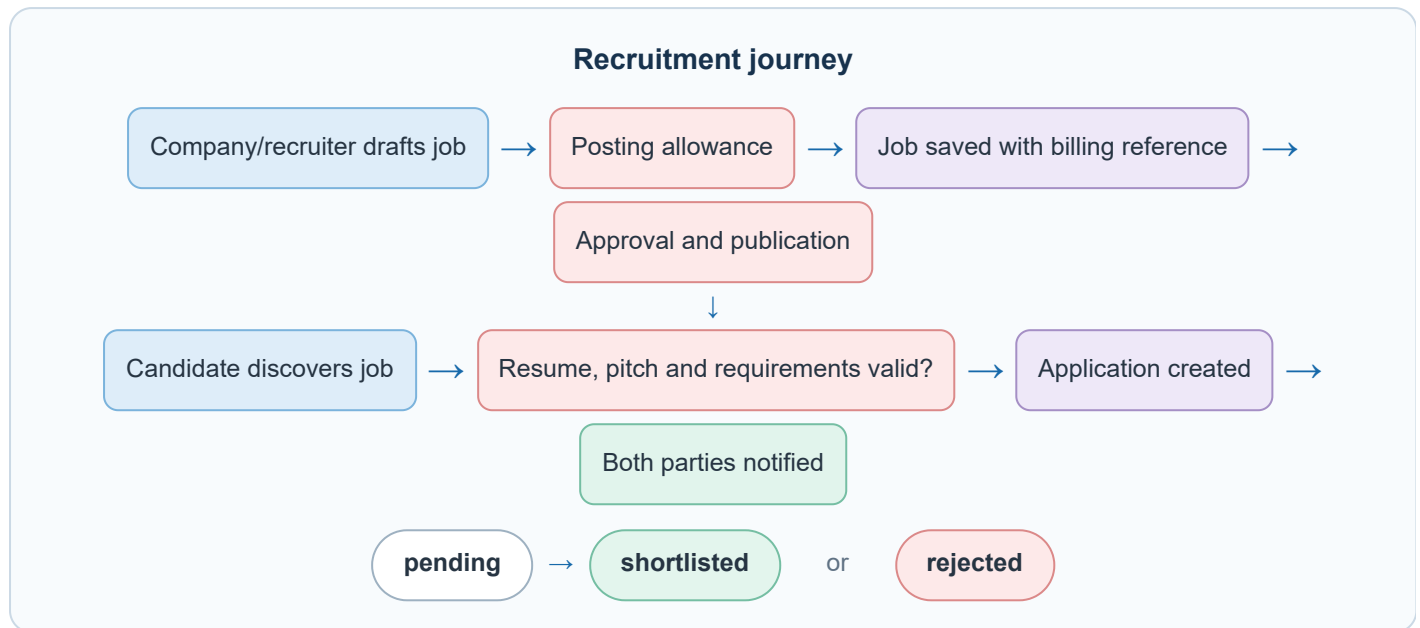
Profiles and resumes



Candidates can maintain personal details, education, experience, skills, awards, resume files, and a generated resume. Recruiters have a dedicated recruiter-account record. Companies have a company profile and can associate recruiters or employees through request and membership workflows.

Public profile routes support candidate, recruiter, and company views. Slugs are used where available, while older routes also accept record or user IDs.

Jobs and applications

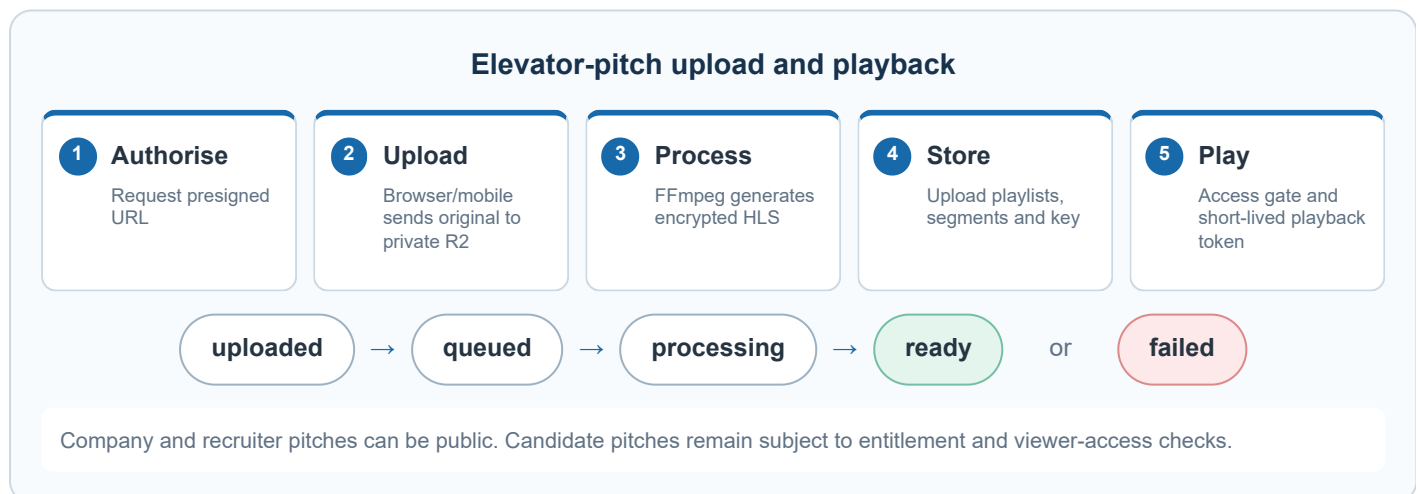


Companies and recruiters can create and manage jobs. Jobs support category, employment type, location type, career level, salary information, publishing and deadline dates, approval status, archiving, and billing attribution.

Candidates can browse, search, receive recommendations, bookmark jobs, apply, and review application history. Recruiters and companies can review applicants and change an application's status to pending, rejected, or shortlisted.

Job creation records its billing source. Credit-backed jobs reference the payment record that supplied the credit. This association is used for durable usage accounting and refund calculations.

Elevator video pitches

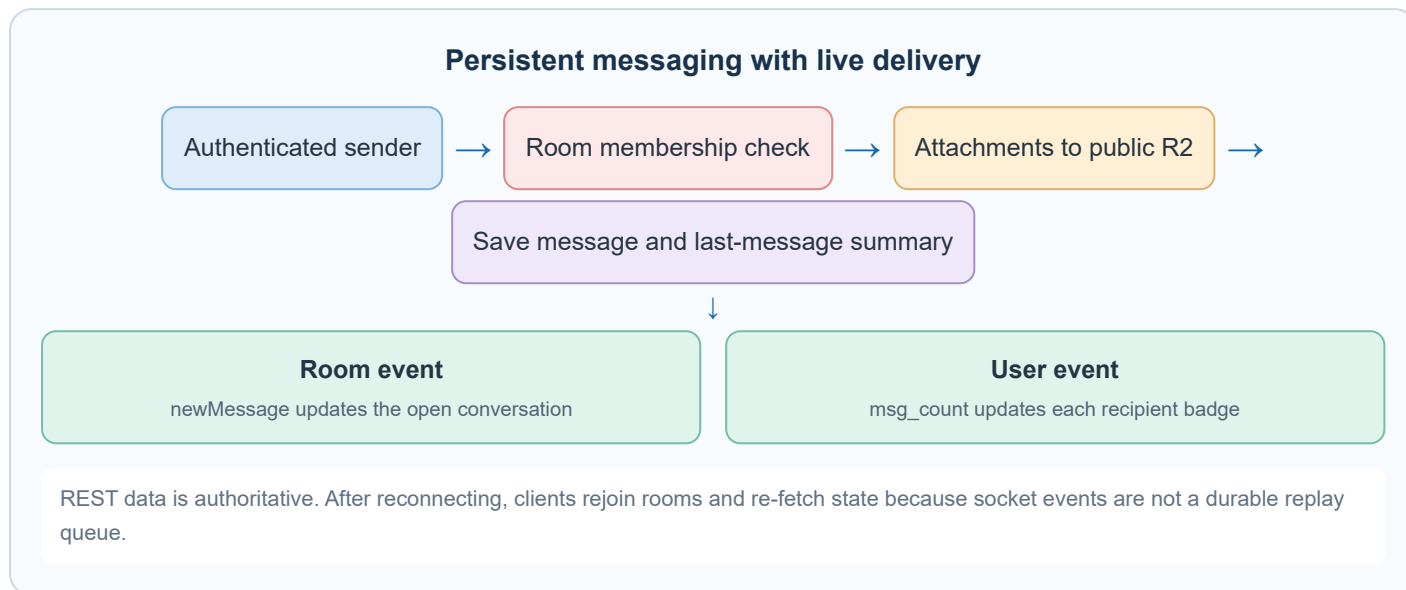


The pitch workflow uses a direct-upload pattern:

1. An authenticated client requests an upload URL.
2. The client uploads the source media to private Cloudflare R2 storage.
3. The client notifies the API that the upload is complete.
4. The backend queues video processing and generates HLS playback assets.
5. Access checks are applied when a viewer requests a stream, segment, key, or playback token.

Company and recruiter pitches can be publicly viewable. Candidate pitches are subject to entitlement and viewer-access rules. Short-lived playback tokens prevent the login JWT from being propagated into HLS playlist and segment URLs.

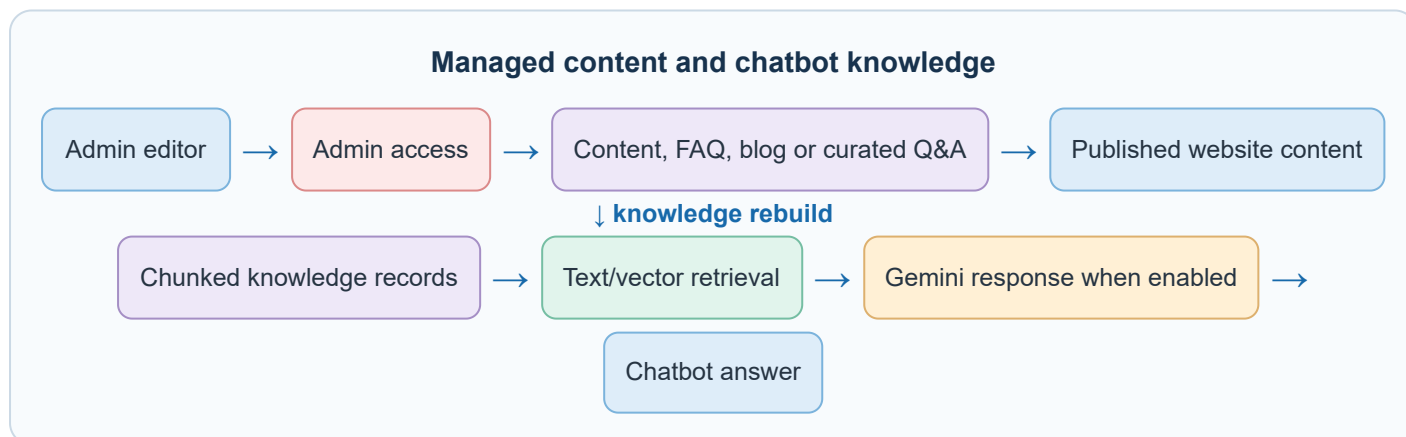
Messaging and notifications



Messages and message rooms are stored in MongoDB. Socket.IO supplies live updates. Clients join a message-room ID for conversation updates and a user ID room for notifications. Notification events currently include `newNotification` and `notificationCountUpdated`.

REST endpoints remain the source of truth for message history, read status, room state, and notification lists. A client should re-fetch after reconnecting because socket delivery is not a permanent queue.

Content, FAQs, blogs, and chatbot



The admin portal manages built-in and custom content pages, FAQs, blogs, scrolling messages, and chatbot question-and-answer entries. The backend ensures that the six built-in content records exist whenever it starts.

The chatbot can answer from curated Q&A and indexed site material. Gemini-backed chat, embeddings, and job-fit features are controlled through environment variables so they can be enabled or disabled independently.

6. Job-post pricing and credits

Catalogue versus completed purchase

SubscriptionPlan catalogue

Current price and allocation for future checkouts. Administrators may edit it.

PaymentInfo snapshot

Amount paid, credits bought, credits used and original transaction. Price edits do not rewrite it.

Catalogue change → new purchases only | Historical payment → remains usable on its original terms

Catalogue behaviour

Company and recruiter job-post products use one-time credits. The public company and recruiter pricing pages read the active catalogue from the API and share the same pricing component.

The source currently displays:

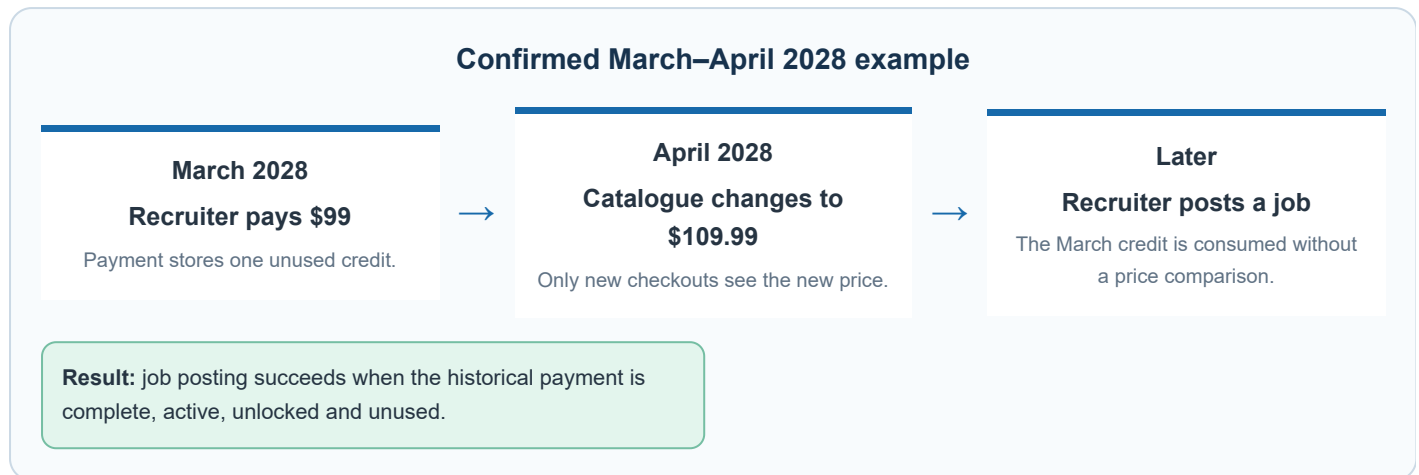
- All job posts free until April 2027.
- Job post rates.
- The more job posts purchased, the more discounts gained.
- Prices apply from April 2027 until March 2028.
- A direction to view the refund policy in Terms and Conditions or search the chatbot.

The default package catalogue used by the initial migration is shown below. Live prices may be changed by an administrator and should always be treated as authoritative.

Package	Default price	Job-post credits
Pay as You Go	\$99.99	1
Basic Plan	\$195.99	2
Premium Basic Plan	\$2,155.99	24
Bronze Plan	\$270.99	3
Premium Bronze Plan	\$2,980.99	36
Silver Plan	\$350.99	4
Premium Silver Plan	\$3,915.99	48
Gold Plan	\$430.99	5
Premium Gold Plan	\$4,839.99	60
Platinum Plan	\$1,199.99	14
Premium Platinum Plan	\$12,319.99	Unlimited

Administrators change catalogue prices and allocations under **Admin > Plan > Edit price**. Company and recruiter catalogue records are separate, so both must be updated when both audiences should receive the same change.

Purchase snapshot and later price changes



When a credit package is paid successfully, the payment record stores:

- the amount actually paid;
- the purchased credit allocation;
- the number of credits used, initially zero;
- the transaction ID and payment method;
- the original purchase timestamp;
- an active plan status; and
- the catalogue plan reference.

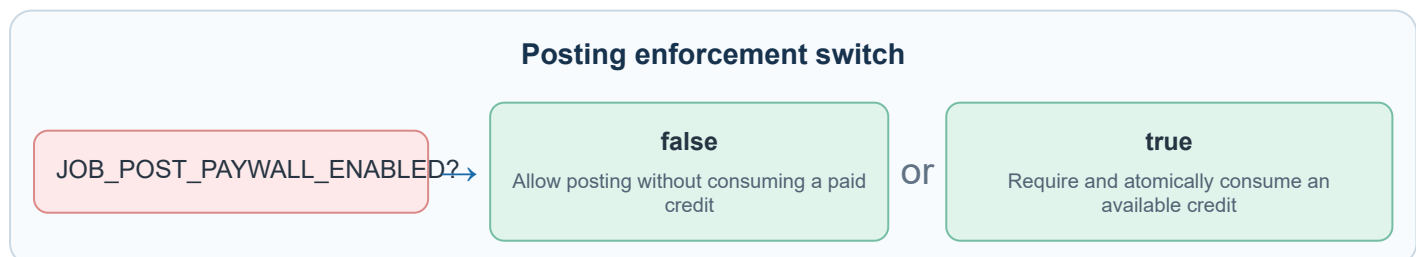
Credit use does not compare the old purchase price with the current catalogue price. The backend looks for a completed, active, non-refunded payment with an unused credit and increments its durable usage counter.

This means a recruiter who buys one credit for \$99 in March 2028 can still use that credit after the catalogue price changes to \$109.99 in April 2028. The \$109.99 price applies to new purchases only. The earlier credit remains valid until it is used, refunded, or administratively deactivated.

Credits do not expire. Purchasing another package adds another payment allocation instead of replacing an earlier balance. The oldest available purchase is consumed first. Deleting a posted job does not return its credit.

Job creation and credit consumption run in the same MongoDB transaction. This prevents two simultaneous requests from spending the last credit twice. Production MongoDB must therefore be Atlas or a replica set with transaction support.

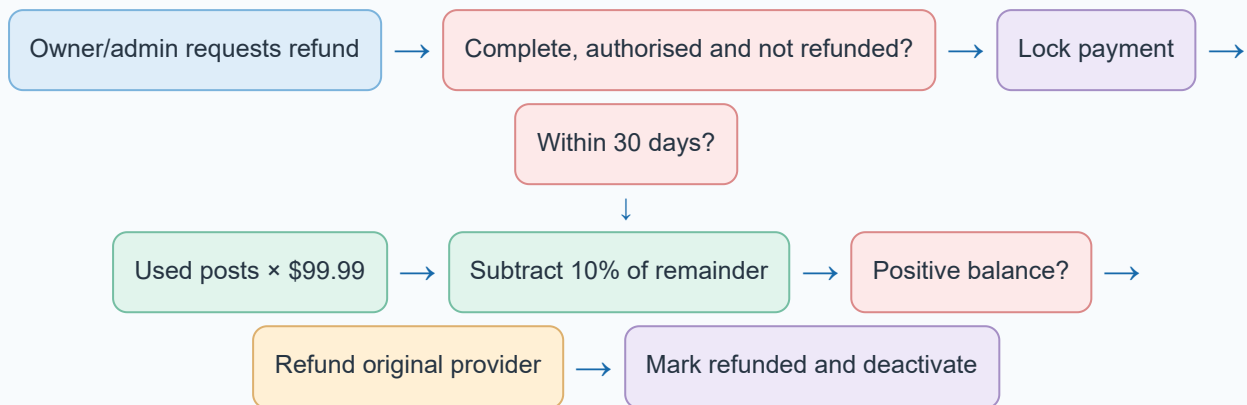
Free-posting switch



`JOB_POST_PAYWALL_ENABLED` controls enforcement. When it is `true`, a company or recruiter needs an available credit. When it is disabled, posting can proceed without consuming a paid credit. Production was recorded with this setting enabled after the September 2026 rollout.

7. Refund rules

Job-package refund pipeline



$$\text{refund} = \max(\text{amount paid} - \text{used-post deductions}, 0) - 10\% \text{ administration fee}$$

If a provider call has an uncertain outcome, keep the refund lock and reconcile in Stripe or PayPal before retrying.

Company and recruiter job-package refunds follow these rules:

- The request must be made no later than 30 days from the original payment timestamp.
- Each job already posted from that purchase is deducted at \$99.99.
- A 10% administration fee is applied to the remaining balance after job deductions.
- Monetary calculations are rounded to cents.
- A refund is not issued when no positive refundable balance remains.
- A refunded payment cannot supply further credits.
- A per-payment lock prevents a refund and credit spend from completing concurrently.

Example for a \$195.99 package with one job already posted:

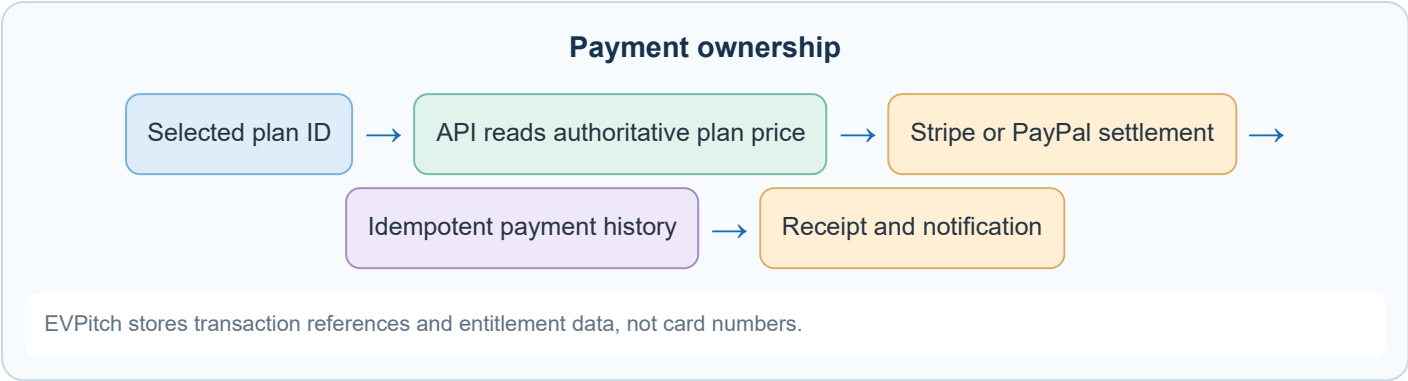
Purchase amount	\$195.99
One job-post deduction	-99.99
Remaining balance	96.00
Administration fee (10%)	-9.60
Refund	\$86.40

The fixed \$99.99 refund deduction is a policy value and is separate from the current Pay as You Go sale price. Changing the PAYG catalogue price does not change the deduction.

If a provider times out after a refund request has been submitted, the payment remains locked for reconciliation. Check Stripe or PayPal before clearing that state; retrying without checking can duplicate a successful provider-side refund.

Candidate monthly and yearly subscriptions retain their existing, separate refund and expiry rules.

8. Payments



Stripe

Stripe purchase sequence				
	Buyer	EVPitch API	Stripe	MongoDB
1	Select plan	Load plan and create Intent from server price	Return client secret	
2	Confirm with Elements		Authorise and settle	
3		Confirmation or signed webhook	Return settled amount and metadata	Create payment once by transaction ID

Credit allocation is copied into Intent metadata so an in-flight Stripe purchase keeps the allocation presented when the Intent was created.

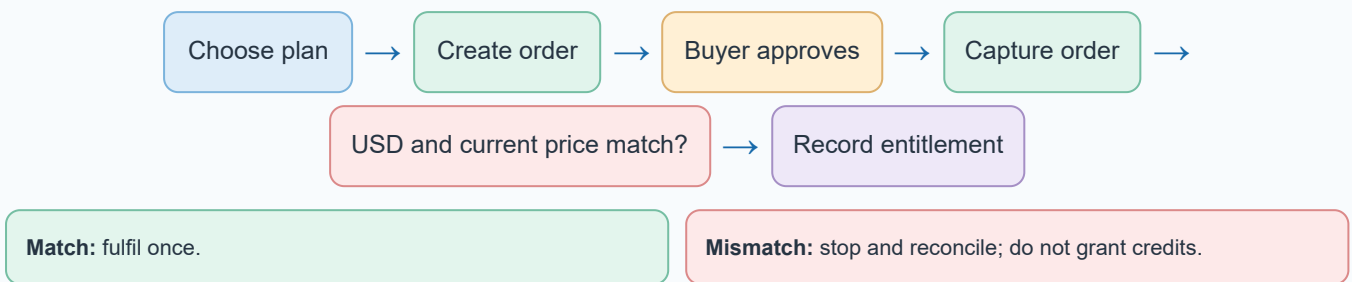
The backend creates Payment Intents using the price read from the catalogue, not an amount supplied by the browser. Credit allocation and audience are copied into Payment Intent metadata. On successful confirmation or webhook fulfilment, the backend records the settled amount and the purchased credit allocation.

The Stripe webhook is registered before JSON body parsing because signature verification requires the raw request body. `STRIPE_WEBHOOK_SECRET` must match the endpoint configured in Stripe.

The website uses Stripe Elements. The Content Security Policy permits Stripe scripts, API connections, frames, and 3-D Secure challenges.

PayPal

PayPal order and capture sequence



Avoid editing a plan while an approved PayPal order for that plan is still awaiting capture.

PayPal runs against the live environment when `NODE_ENV=production`; all other environments use the sandbox. At capture, the backend checks that the currency is USD and that the captured amount matches the selected catalogue entry.

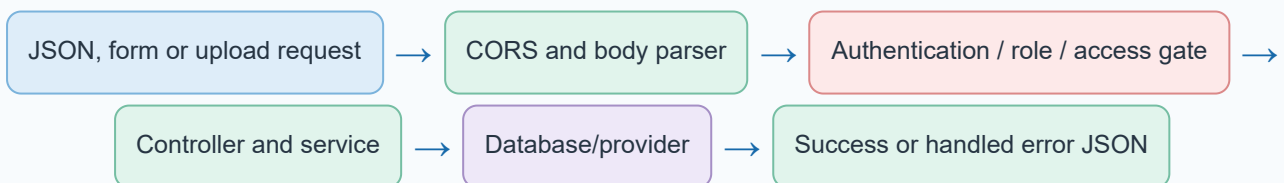
For a clean operational change, avoid editing a price while a PayPal order for that exact catalogue record is awaiting capture. A completed purchase is unaffected by later price changes.

Idempotency and history

Credit payments have a unique transaction index. Stripe confirmation, webhook fulfilment, and repeated callbacks return the existing payment when the transaction has already been recorded. Historical payments keep their original amount and allocation when an administrator edits a plan.

9. API conventions

API request lifecycle



Base addresses

```
Service check: GET /
Health check:  GET /health
API base:      /api/v1
```

Successful controller responses generally follow:

```
{
  "success": true,
  "message": "Optional description",
  "data": {}
}
```

Handled errors generally follow:

```
{
  "success": false,
  "message": "Readable error message",
  "errorSources": [
    { "path": "", "message": "Readable error message" }
  ]
}
```

Protected requests use:

```
Authorization: Bearer <access-token>
```

JSON and URL-encoded request bodies are limited to 25 MB. File-upload limits are defined separately by the relevant Multer middleware and storage workflow.

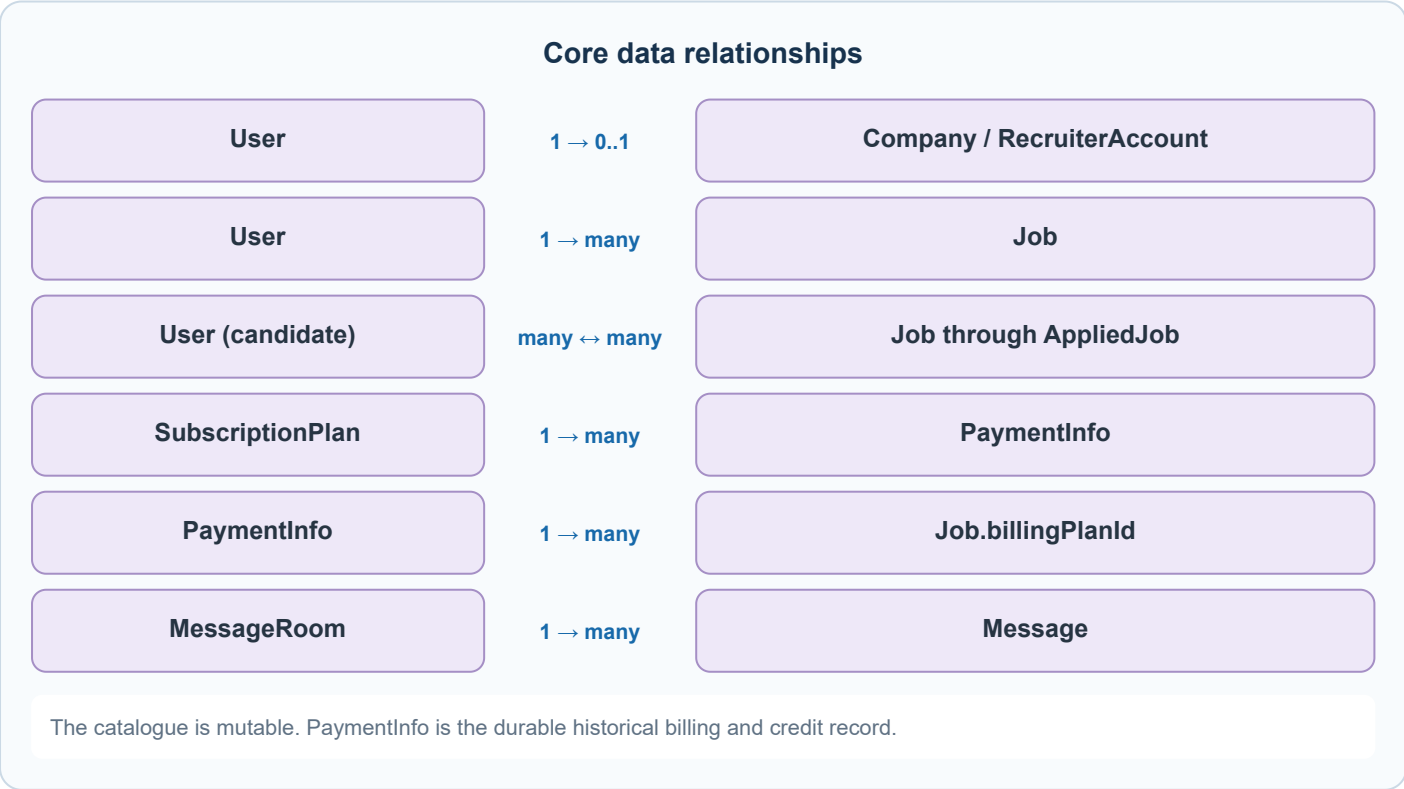
Route groups

API prefix	Main responsibility
<code>/api/v1/user/*</code> and selected <code>/api/v1/*</code> routes	Registration, login, OTP, recovery, users, profiles, and search
<code>/api/v1/jobs*</code>	Jobs, recommendations, usage, AI fit, archive, and approval
<code>/api/v1/category/*</code>	Job categories
<code>/api/v1/subscription/plans*</code>	Candidate plans and company/recruiter credit catalogue
<code>/api/v1/payments/*</code>	Stripe, PayPal, payment history, and refunds
<code>/api/v1/applied-jobs/*</code>	Applications and applicant status
<code>/api/v1/company/*</code>	Company profile and recruiter/employee relationships
<code>/api/v1/recruiter/*</code>	Recruiter accounts
<code>/api/v1/elevator-pitch/*</code>	Direct upload, processing state, playback, and deletion
<code>/api/v1/resume/*</code> and <code>/api/v1/create-resume/*</code>	Uploaded and generated resumes
<code>/api/v1/message-room/*</code> and <code>/api/v1/message/*</code>	Conversations and messages

API prefix	Main responsibility
/api/v1/notifications/*	Notification retrieval and read state
/api/v1/following/*	Recruiter following
/api/v1/bookmarks/*	Saved jobs
/api/v1/content/*	Terms, policies, built-in pages, and custom pages
/api/v1/blogs/*	Blog content
/api/v1/faqs/*	Frequently asked questions
/api/v1/chatbot/*	Chat, knowledge rebuild, and curated Q&A
/api/v1/scrolling-info/*	Role-specific scrolling messages
/api/v1/newsletter/*	Subscribers and newsletter sending
/api/v1/skill/*, /language/*, /university/*, /coudency/*	Reference data and bulk imports
/api/v1/countries/*	Country and city data
/api/v1/admin/stats	Administration dashboard statistics

The route files in `src/routes` are the definitive endpoint register. Authentication varies by operation; integrations must verify the route declaration before relying on an endpoint as public or protected.

10. Data model summary

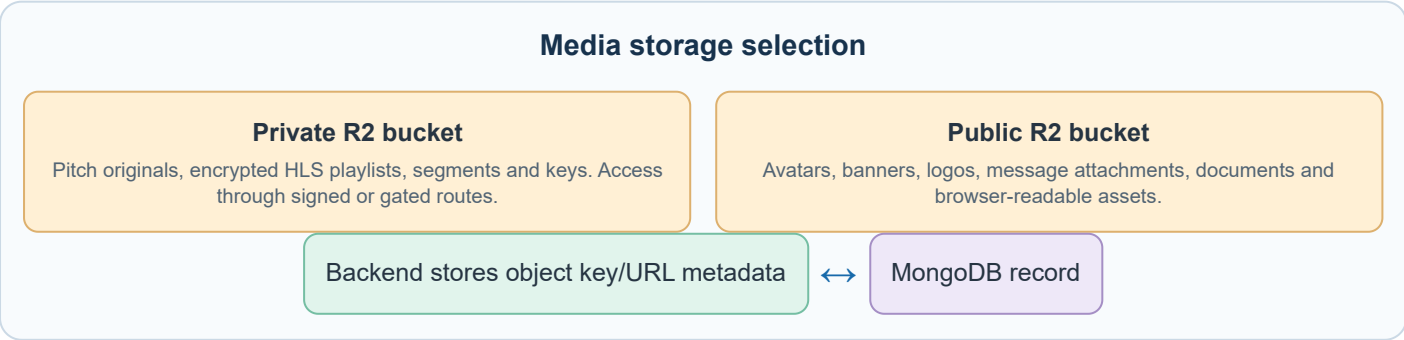


Collection area	Important records and relationships
Identity	User is the login identity and role owner. Company and recruiter records reference a user.
Profiles	Education, experience, awards, skills, generated resumes, uploaded resumes, and elevator pitches link back to users.
Recruitment	Jobs reference their owner, category, approval state, and billing source. Applications connect users and jobs. Bookmarks and following records capture user relationships.
Billing	Subscription plans are the editable catalogue. Payment records are immutable purchase history plus mutable usage/refund state. Jobs can reference the payment that funded them.
Communication	Message rooms contain participants; messages reference rooms and read state. Notifications target users.
Publishing	Content, FAQs, blogs, scrolling information, newsletter subscriptions, and chatbot records support managed site content.

Important billing fields:

Field	Meaning
SubscriptionPlan.price	Current sale price for future purchases
SubscriptionPlan.jobPostCredits	Current allocation; null means unlimited
PaymentInfo.amount	Amount paid for that historical transaction
PaymentInfo.jobPostCredits	Allocation purchased in that transaction
PaymentInfo.jobPostsUsed	Durable number of credits consumed
PaymentInfo.duration	credits for non-expiring job packages
PaymentInfo.paymentStatus	Pending, complete, failed, or refunded
PaymentInfo.planStatus	Active or deactivated entitlement
Job.billingPlanId	Payment record that supplied the job credit

11. Storage and media



EVPitch uses two Cloudflare R2 storage patterns:

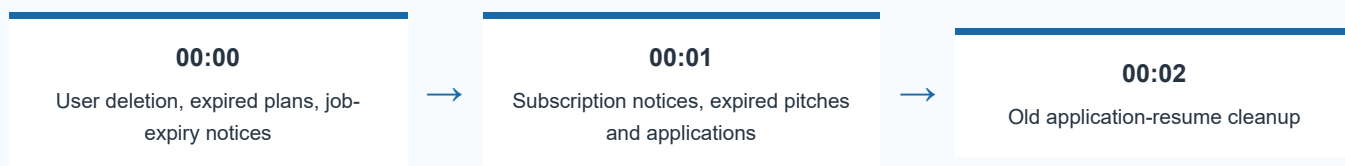
- Private storage holds original pitch media, HLS playlists, segments, and encryption keys. Access is granted through signed URLs or controlled streaming endpoints.
- Public storage holds browser-readable assets such as avatars, banners, logos, images, documents, and selected short videos. MongoDB stores stable public URLs.

Cloudinary configuration remains in the backend for workflows that still use it. Local `uploads` are exposed by Express for legacy files, but new media should use the appropriate R2 service.

Temporary upload files are removed after public R2 upload. Video processing requires FFmpeg and FFprobe to be installed and reachable by the backend process.

12. Scheduled and startup work

Daily maintenance timeline — server local time



No timezone is passed to node-cron, so the production host timezone controls these execution times.

On startup, the backend connects to MongoDB, creates missing built-in content, and aligns legacy job-approval fields.

The server schedules daily maintenance:

Time	Work
00:00 server time	Delete eligible deactivated users, update expired plans, and notify recruiters about job expiry
00:01 server time	Notify expired subscriptions, remove expired elevator pitches, and purge expired job applications
00:02 server time	Remove old application resume files

Cron uses the server's local timezone because no timezone is supplied in the schedule. Production hosts should have a deliberate and documented timezone setting.

13. Environment configuration

Create environment files on the relevant host. Never commit live values.

Backend essentials

Variable	Purpose
<code>NODE_ENV</code>	Selects production behaviour, including live PayPal
<code>PORT</code>	HTTP and Socket.IO listening port; defaults to 5000

Variable	Purpose
MONGO_URI	MongoDB connection string
JWT_ACCESS_SECRET , JWT_REFRESH_SECRET	JWT signing secrets
JWT_ACCESS_EXPIRES_IN , JWT_REFRESH_EXPIRES_IN	Token lifetimes
OTP_SECRET	OTP signing or verification secret
BCRYPT_SALT_ROUNDS	Password hashing cost
SERVER_URL	Public backend origin used when generating links
JOB_POST_PAYWALL_ENABLED	Enables enforcement and consumption of job credits

Payments

Variable	Purpose
STRIPE_SECRET_KEY	Server-side Stripe API key
STRIPE_PUBLISHABLE_KEY	Publishable key exposed through the config endpoint where used
STRIPE_WEBHOOK_SECRET	Stripe webhook signature secret
PAYPAL_CLIENT_ID , PAYPAL_CLIENT_SECRET	Server-side PayPal credentials

Email

Variable	Purpose
SMTP_HOST , SMTP_PORT	SMTP server connection
APP_USER , APP_PASSWORD	SMTP authentication
EMAIL_FROM , NO_REPLY_EMAIL	General sender addresses
CANDIDATE_EMAIL_FROM	Candidate-specific sender identity
RECRUITER_EMAIL_FROM	Recruiter-specific sender identity
COMPANY_EMAIL_FROM	Company-specific sender identity

Media storage and playback

Variable	Purpose
R2_ACCOUNT_ID , R2_ACCESS_KEY_ID , R2_SECRET_ACCESS_KEY	Private R2 access
R2_BUCKET_NAME , R2_PRIVATE_BASE	Private bucket and base address

Variable	Purpose
R2_PUBLIC_ACCOUNT_ID , R2_PUBLIC_ACCESS_KEY_ID , R2_PUBLIC_SECRET_ACCESS_KEY	Public R2 access; private credentials are fallback values
R2_PUBLIC_BUCKET_NAME , R2_PUBLIC_BASE	Public asset bucket and public origin
AWS_BUCKET_NAME , AWS_BUCKET_VISIBILITY	Compatibility settings used by older storage paths
PITCH_PLAYBACK_SECRET	Signs short-lived pitch playback tokens
CLOUDINARY_CLOUD_NAME , CLOUDINARY_API_KEY , CLOUDINARY_API_SECRET	Legacy/alternate Cloudinary integration

AI and search

Variable	Purpose
GEMINI_API_KEY	Gemini API access
GEMINI_CHAT_MODEL	Chatbot model selection
GEMINI_EMBED_MODEL	Embedding model selection
GEMINI_THINKING_LEVEL	Optional reasoning configuration
JOB_FIT_AI	Enables AI job-fit analysis
JOB_EMBEDDINGS	Enables job embedding generation
MONGODB_VECTOR_INDEX	MongoDB Atlas vector index name

Public website

Variable	Purpose
NEXT_PUBLIC_BASE_URL	API base including /api/v1
NEXT_PUBLIC_API_BASE_URL	Chatbot or alternate API base where configured
NEXT_PUBLIC_SOCKET_URL	Socket.IO server origin
NEXT_PUBLIC_SITE_URL , NEXT_PUBLIC_APP_URL , NEXT_PUBLIC_BASE_SHARE_URL	Website and share-link origins
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY	Stripe Elements publishable key
NEXT_PUBLIC_PAYPAL_CLIENT_ID	PayPal browser SDK client ID
NEXTAUTH_SECRET	NextAuth session signing secret required in production

Admin portal

Variable	Purpose
NEXT_PUBLIC_BASE_URL	API base including /api/v1
NEXT_PUBLIC_SITE_URL	Admin portal origin where required
NEXTAUTH_SECRET	Admin session signing secret

14. Local development

Prerequisites

- A current Node.js LTS release compatible with Next.js 15 and the package lockfiles.
- npm.
- MongoDB Atlas or a local replica set when testing credit transactions.
- Flutter SDK compatible with Dart ^3.9.2 for mobile work.
- FFmpeg and FFprobe for pitch processing.
- Sandbox Stripe, PayPal, R2, SMTP, and Gemini credentials for the workflows being tested.

Backend

```
cd evpitch-backend
npm ci
npm run dev
```

The API defaults to port 5000. Confirm GET /health before starting client applications.

Public website

```
cd evpitch-frontend
npm ci
npm run dev
```

Admin portal

```
cd evpitch-admin
npm ci
npm run dev
```

Flutter

```
cd flutter_giveandtake_getx
flutter pub get
flutter run
```

Do not use live payment credentials for local testing. The mobile source contains fixed production and sandbox addresses in some older payment paths; review those constants before running a payment test.

15. Build and test commands

Verification layers

End-to-end: checkout, post, apply, refund

Integration: API, MongoDB transactions, providers and storage

Component: web pages, admin forms and mobile controllers

Unit: policies, expiry, refund calculations and validation

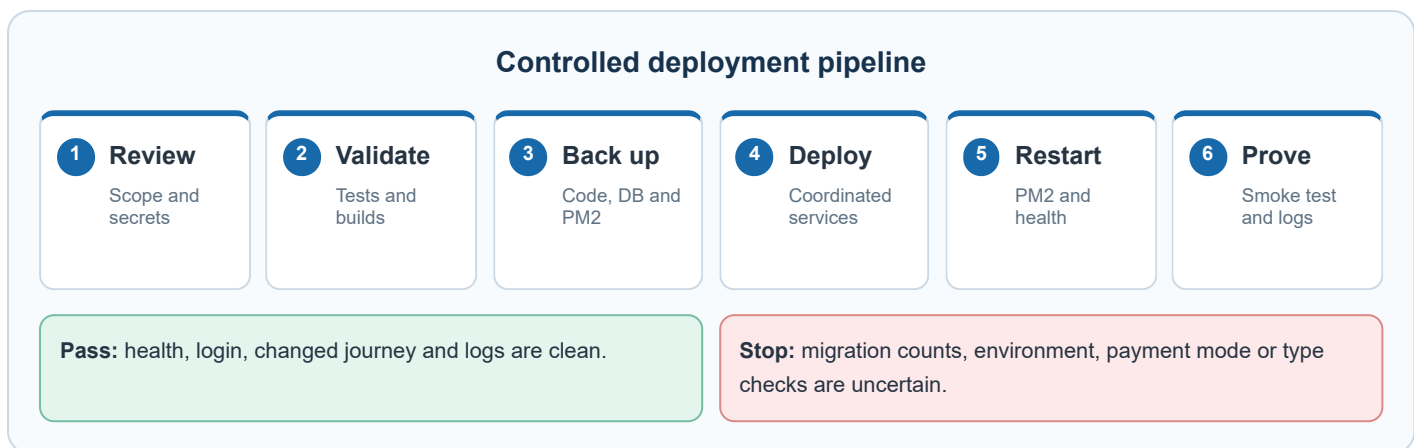
A successful Next.js build is not a substitute for an explicit TypeScript check while build-error ignoring remains enabled.

Component	Build	Tests/checks
Backend	npm run build	npm test ; npm run lint
Public website	npm run build	npx tsc --noEmit --incremental false
Admin portal	npm run build	npx tsc --noEmit --incremental false
Flutter	flutter build <target>	flutter test ; flutter analyze

Backend tests cover the job-package catalogue, non-expiry, purchase accumulation, durable usage, unlimited allocations, candidate expiry, refund amounts, and the exact 30-day boundary.

At the time of this handover, the focused backend job-package suites pass all 15 tests. The public website's full TypeScript check still reports existing errors outside the pricing component. The Next.js configuration currently allows production builds to continue despite lint and TypeScript errors, so a successful website build alone should not be treated as a clean type check.

16. Deployment and release procedure



The recorded production layout uses PM2, with the backend and admin portal under `/var/www/api` and `/var/www/admin`. Confirm actual paths and process names on the host before issuing commands; they are operational details, not application constants.

For a routine code release:

1. Review the commits in each affected repository and confirm that environment files are not included.
2. Back up the affected code and database collections.
3. Install exact dependencies with `npm ci` or the established lockfile-based process.
4. Run backend tests and compile the backend.
5. Run explicit TypeScript checks and production builds for each Next.js application.
6. Deploy backend and clients as one coordinated release when API contracts changed.
7. Restart the relevant PM2 processes.
8. Check `/health`, login, the changed workflow, and browser/server logs.
9. Where payments changed, run a sandbox checkout, post, and refund smoke test before approving the release.

Database migrations

The initial non-expiring job-package rollout used:

```
npm run migrate-job-packages
npm run migrate-job-packages -- --apply
```

The first command is a preview. The second writes changes. The full migration reapplies the original ten package prices and must not be used for routine catalogue editing after administrators have customised prices.

The focused PAYG restoration uses:

```
npm run restore-job-payg
npm run restore-job-payg -- --apply
```

Always preview, back up `subscriptionplans`, `paymentinfos`, and `jobs`, and use a maintenance window before an applying migration.

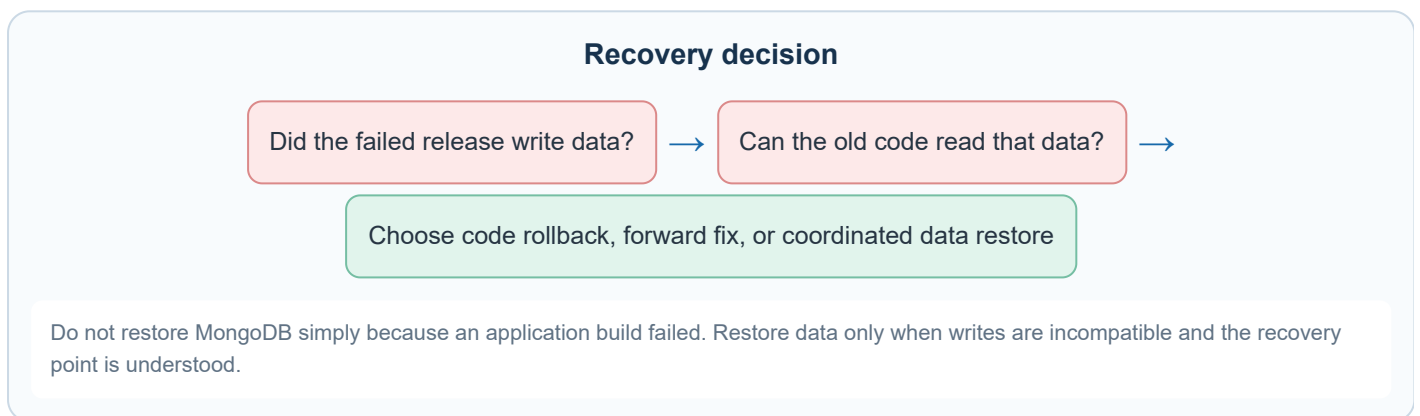
Routine price changes

A normal price change requires no migration and no code deployment:

1. Open **Admin > Plan**.
2. Locate the package and audience.
3. Select **Edit price**.
4. Enter the new USD price and confirm the credit allocation.
5. Save the change.
6. Repeat for the other audience when appropriate.
7. Verify the public company and recruiter pricing pages.

Existing completed purchases must not be edited. Their recorded amounts and credits remain valid at the terms on which they were bought.

17. Backup and recovery



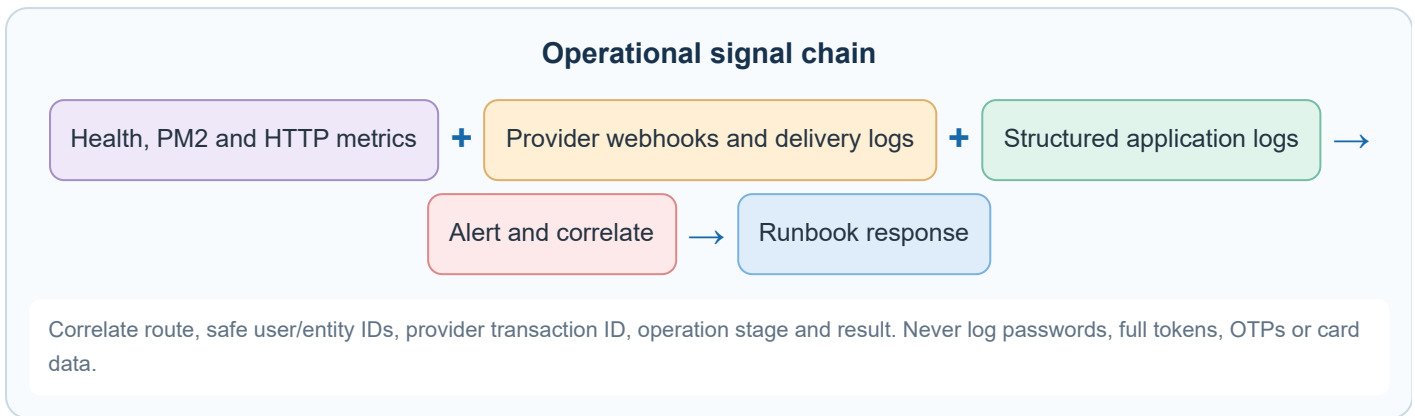
Before billing, migration, or deletion work, back up at least:

- `subscriptionplans`;
- `paymentinfos`;
- `jobs`;
- `users`;
- `appliedjobs`;
- content and media metadata affected by the release; and
- the active PM2 configuration and deployed code.

MongoDB backups should be restorable and periodically tested. R2 lifecycle and recovery arrangements should be documented separately in the infrastructure account because deleting an object can leave a valid MongoDB URL pointing to missing media.

If a release fails, restore the previous code and environment first. Restore database collections only when the failed release wrote incompatible data; rolling back code against already migrated data can be more damaging than leaving the new schema in place.

18. Monitoring and operational checks



At minimum, monitor:

- API availability and `/health` response time;
- PM2 process restarts and memory usage;
- MongoDB connection errors, storage growth, and transaction failures;
- HTTP 4xx/5xx rates;
- Stripe and PayPal webhook failures;
- SMTP delivery failures;
- R2 upload and playback failures;
- FFmpeg queue failures or pitches stuck in `queued` or `processing`;
- cron task completion;
- failed login, OTP, and password-reset patterns; and
- socket connection and reconnect errors.

Operational logs must not include passwords, full JWTs, payment credentials, card details, or private playback keys.

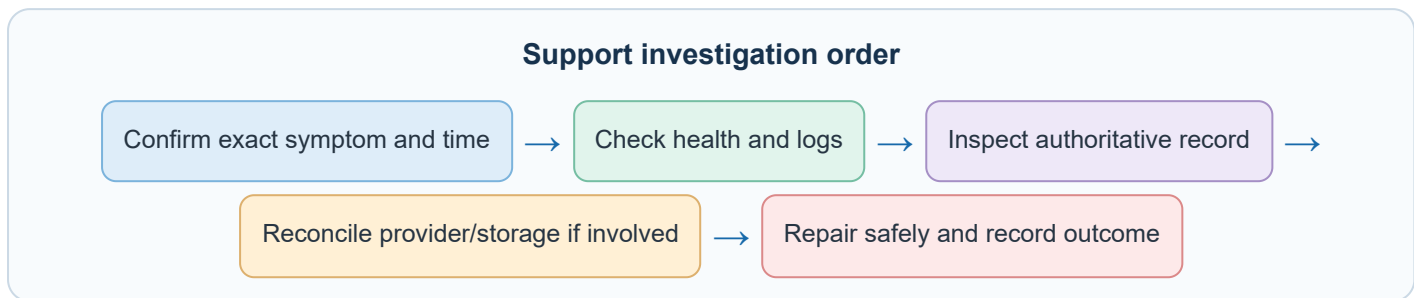
19. Security notes and known technical debt

The following items deserve priority in future maintenance:

1. Restrict API and Socket.IO CORS to approved production and staging origins. The current server configuration allows all origins.
2. Review every route for authentication and role enforcement. Several older mutation and administrative-looking routes do not declare `protect` or `isAdmin` at route level.
3. Authenticate Socket.IO connections and authorise room membership on the server. A client-provided room or user ID should not be trusted by itself.
4. Resolve the public website TypeScript backlog and remove `ignoreBuildErrors` and `ignoreDuringBuilds` once clean.
5. Remove test credentials and obsolete local addresses from source documentation and mobile code. Rotate any credential that may previously have been shared.
6. Move fixed mobile API/payment addresses to build-time environment flavours for development, staging, and production.
7. Add request validation consistently to payment, identity, content, and upload endpoints.
8. Add rate limiting to login, OTP, password recovery, chatbot, contact, and other abuse-sensitive public endpoints.
9. Confirm that production cookies, proxy headers, TLS, CSP, and secrets follow the live hosting design.
10. Add automated integration coverage for provider webhooks, MongoDB transaction conflicts, direct uploads, socket authorisation, and full refund reconciliation.

These notes describe the present source tree and should be treated as a maintenance list, not as evidence that a production incident has occurred.

20. Support runbook



A user cannot post a job

1. Check `JOB_POST_PAYWALL_ENABLED`.
2. Confirm the user role and account status.
3. Find completed, active `credits` payments for the user.
4. Compare `jobPostsUsed` with `jobPostCredits`; `null` means unlimited.
5. Confirm the payment is not refunded or locked for refund processing.
6. Check API logs for transaction or MongoDB replica-set errors.
7. Do not manually change a catalogue price or create a fake payment to repair a historical purchase.

A historical credit fails after a price change

The current price should not affect credit use. Check the historical payment's `paymentStatus`, `planStatus`, `refundProcessing`, `jobPostCredits`, and `jobPostsUsed`. A price mismatch during posting would indicate a regression because the credit-consumption service does not read the catalogue price.

A price is different on the two public pages

Company and recruiter plans are separate records. Edit and verify both audience entries in Admin > Plan. Confirm that the website is using the correct API environment and clear any intermediary cache if the API already returns the new price.

A payment completed but credits are missing

1. Check the provider dashboard for the final status.
2. Locate the transaction ID in `paymentinfos`.
3. Inspect webhook and confirmation logs.
4. Confirm that the plan metadata contains `valid=credits` and a valid allocation.
5. Reconcile idempotently by transaction ID; never insert a second payment blindly.

A refund appears stuck

Check the provider before removing `refundProcessing`. A network timeout can occur after the provider has accepted a refund. Record the reconciliation outcome and provider refund ID before changing local state.

A video will not play

Confirm the pitch status is `ready`, the HLS objects exist in the private bucket, the playback or access token is valid, the viewer has permission, and the CSP permits the configured API/media origin. Check FFmpeg and R2 logs if processing never reached `ready`.

21. Acceptance record for the September 2026 pricing work

The following work is complete in the current source and recorded deployment notes:

- Company and recruiter packages use non-expiring job-post credits.
- Existing purchases retain their paid amount, allocation, and remaining credits after catalogue edits.
- PAYG is present for both audiences and can be edited through Admin > Plan.
- Public company and recruiter pricing pages use the shared rate-card component.
- The revised pricing heading and refund-policy wording have been added to that shared component.
- The 30-day refund window, fixed \$99.99 used-post deduction, and 10% administration fee are implemented.
- Credit consumption and job creation are transactional.
- The focused backend package and PAYG tests pass all 15 cases.

The last recorded live deployment was 9 September 2026. No real purchase or refund was made during that deployment verification. A sandbox end-to-end checkout, post, and refund remains the recommended acceptance test whenever payment code or provider configuration changes.

22. Ownership and handover checklist



The system owner should retain controlled access to:

- source repositories and release branches;
- production and staging environment variables;
- MongoDB Atlas;
- Cloudflare R2;
- Stripe and PayPal;
- SMTP/email provider;
- Gemini/AI provider;
- domain and DNS settings;
- hosting/VPS and PM2;
- Apple and Google developer accounts for mobile releases; and
- backups, monitoring, and incident contacts.

Before a new maintainer takes responsibility, confirm that they can build each component, reach the staging services, deploy without receiving production secrets in chat, restore a database backup, inspect payment-provider events, and follow the

support runbook above.

End of technical handover